

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

Aim: Simulate smoothing and sharpening operation on images using spatial filters.

Theory:

Smoothing and sharpening are common image enhancement techniques used to modify the spatial characteristics of an image.

Smoothing:

Smoothing, also known as blurring, is used to reduce noise and remove fine details in an image. It works by applying a low-pass filter that averages the pixel values within a neighborhood, thus producing a blurred effect.

Sharpening:

Sharpening is used to enhance the edges and fine details in an image. It works by applying a high-pass filter that amplifies the differences between neighboring pixel values, thus increasing the contrast and emphasizing edges.

Explanation:

We import the required modules, including cv2 for image processing, numpy for array operations, and matplotlib.pyplot for visualization.

The `apply_smoothing_filter` function applies a smoothing filter to the image using `cv2.blur` and returns the smoothed image.

The `apply_sharpening_filter` function creates a sharpening kernel using a predefined matrix, and then applies the kernel to the image using `cv2.filter2D` to produce the sharpened image. In the main part of the program, we load the input image using `cv2.imread`. We apply the smoothing filter to the input image by calling the `apply_smoothing_filter` function with a specified kernel size.

We apply the sharpening filter to the input image by calling the `apply_sharpening_filter` function.

The original image, smoothed image, and sharpened image are displayed side by side using `cv2.imshow` and `np.hstack`.

The filtered images are saved to separate files using `cv2.imwrite`.

You can replace the `image_path` variable with the path to your input image. Also, you can modify the parameters of the filters (kernel size for smoothing) based on your requirements.

Remember to have OpenCV and matplotlib installed (`pip install opencv-python matplotlib`) before running this program.

Programme:

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

```

image=cv2.imread(r"C:\Users\bhoga\Downloads\Final_Image.png")
#mean filter, in weighted mean we have to count the additional number that I have given as weight in sort sum
of all the values
#as kernel size increases smoothening will increase
weighted_kernel=np.asarray([[1,1,1],[1,2,1],[1,1,1]],dtype=np.float32)
weighted_kernel=weighted_kernel/10
weighted_mean=cv2.filter2D(image,-1,weighted_kernel)
kernel2=np.asarray([[1,1,1,1,1],[1,1,1,1,1],[1,1,2,1,1],[1,1,1,1,1],[1,1,1,1,1]],dtype=np.float32)
kernel2=kernel2/26
kernel2_mean=cv2.filter2D(image,-1,kernel2)

plt.figure(figsize=(12,10))
plt.subplot(1,3,1)
plt.imshow(image,cmap='gray')
plt.title("Original image")
plt.axis('off')
plt.subplot(1,3,2)
plt.imshow(cv2.cvtColor(weighted_mean,cv2.COLOR_BGR2RGB))
plt.title("Weighted Mean Filter")
plt.axis('off')
plt.subplot(1,3,3)
plt.imshow(cv2.cvtColor(kernel2_mean,cv2.COLOR_BGR2RGB))
plt.title("Image with 5x5 kernel")
plt.axis('off')
plt.show()
#gaussina filter
#here the image smoothing depends on value of sigma
g0=cv2.GaussianBlur(image,(5,5),sigmaX=0)

```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

```

g1=cv2.GaussianBlur(image,(5,5),sigmaX=1)
plt.figure(figsize=(8,6))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(g0,cv2.COLOR_BGR2RGB))
plt.title("sigma=0")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(g1,cv2.COLOR_BGR2RGB))
plt.title("sigma=1")
plt.axis('off')
plt.show()

```

#sharpening Filters

#1.Laplacian filter

```

laplacian=cv2.Laplacian(image,cv2.CV_64F)
#CV_64F allows all positive negative and decimal values
laplacian_abs=cv2.convertScaleAbs(laplacian)
#convert signed laplacian values to positive magnitudes and 8-bit formates for visualization
plt.imshow(laplacian_abs,cmap='gray')
plt.title("Laplacian edge detection")
plt.axis('off')
plt.show()

```

#roberts filter

```

roberts_x=np.asarray([[1,0],[0,-1]],dtype=np.float32)
roberts_y=np.asarray([[0,1],[-1,0]],dtype=np.float32)

gx=cv2.filter2D(image.astype(np.float32),cv2.CV_32F,roberts_x)

```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

```
gy=cv2.filter2D(image.astype(np.float32),cv2.CV_32F,roberts_y)
```

```
roberts=cv2.magnitude(gx,gy)
```

```
roberts=cv2.normalize(roberts,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
```

```
plt.imshow(roberts,cmap='gray')
```

```
plt.title('Roberts Edge Detection')
```

```
plt.axis('off')
```

```
plt.show()
```

#Prewitt operator

```
prewitt_x=np.asarray([[ -1,0,1],[ -1,0,1],[ -1,0,1]],dtype=np.float32)
```

```
prewitt_y=np.asarray([[ -1,-1,-1],[ 0,0,0],[ 1,1,1]],dtype=np.float32)
```

```
px=cv2.filter2D(image.astype(np.float32),cv2.CV_32F,prewitt_x)
```

```
py=cv2.filter2D(image.astype(np.float32),cv2.CV_32F,prewitt_y)
```

```
prewitt=cv2.magnitude(px,py)
```

```
prewitt=cv2.normalize(prewitt,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
```

```
plt.imshow(prewitt,cmap='gray')
```

```
plt.show()
```

Output:

Original image Weighted Mean Filter image with 5x5 kernel



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

sigma=0



sigma=1

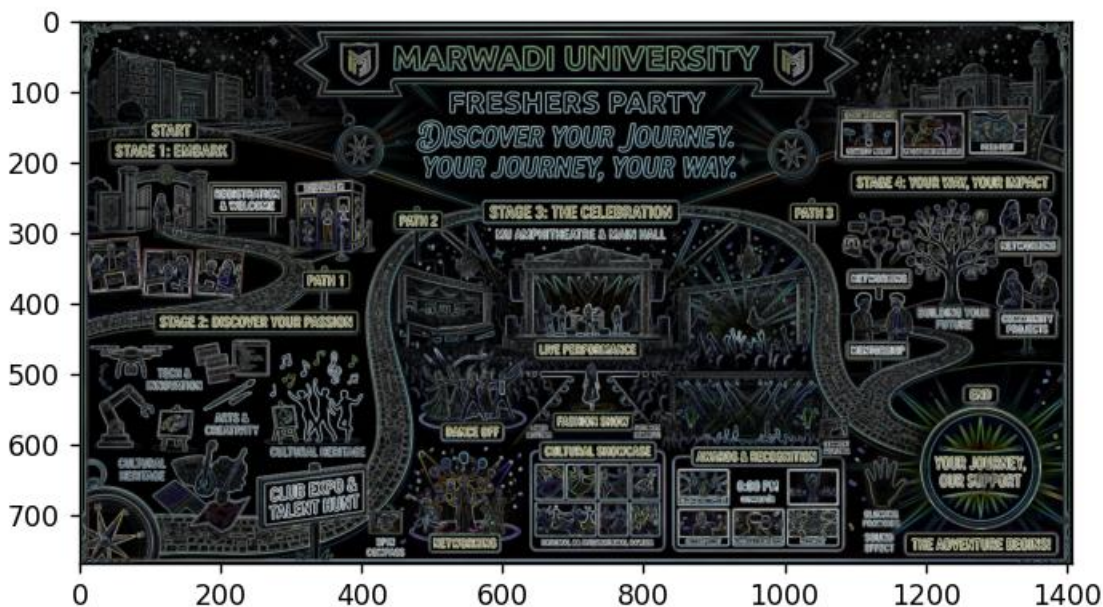
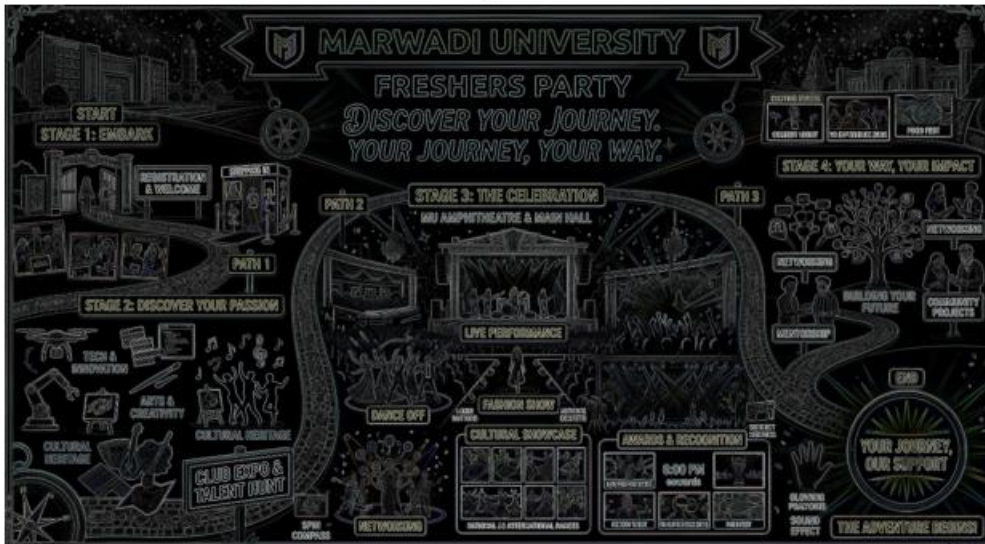


Laplacian edge detection



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

Roberts Edge Detection



Conclusion:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	Aim: Simulate smoothing and sharpening operation on images using spatial filters.	
Experiment-9	Date:04-10-2026	Enrollment No:92400133120

In the above experiment different smoothing and sharpening filters were applied and their outputs were observed through plots. In few filters the effect of parameters were also observed and their effect on image were also observed. This difference was visualized by plotting these different images together.

Github link= https://github.com/Ira1250B/Digital-_Signal_and_Image_Processing